Лабораторная работа №8 "Тестирование фронтенд-приложения"

Срок сдачи: четверг, 14 мая 2026, 23:59

1. ЦЕЛЬ РАБОТЫ

Освоение основ тестирования JavaScript кода и эффективного использования инструментов разработчика для отладки и анализа производительности веб-приложений.

2. НЕОБХОДИМЫЕ ЗНАНИЯ

Для успешного выполнения работы студент должен знать:

- JavaScript ES6+ (функции, объекты, массивы)
- Базовые понятия тестирования
- React компоненты и JSX (из ЛР7)
- HTML/CSS и структуру DOM

3. ЗАДАЧИ РАБОТЫ

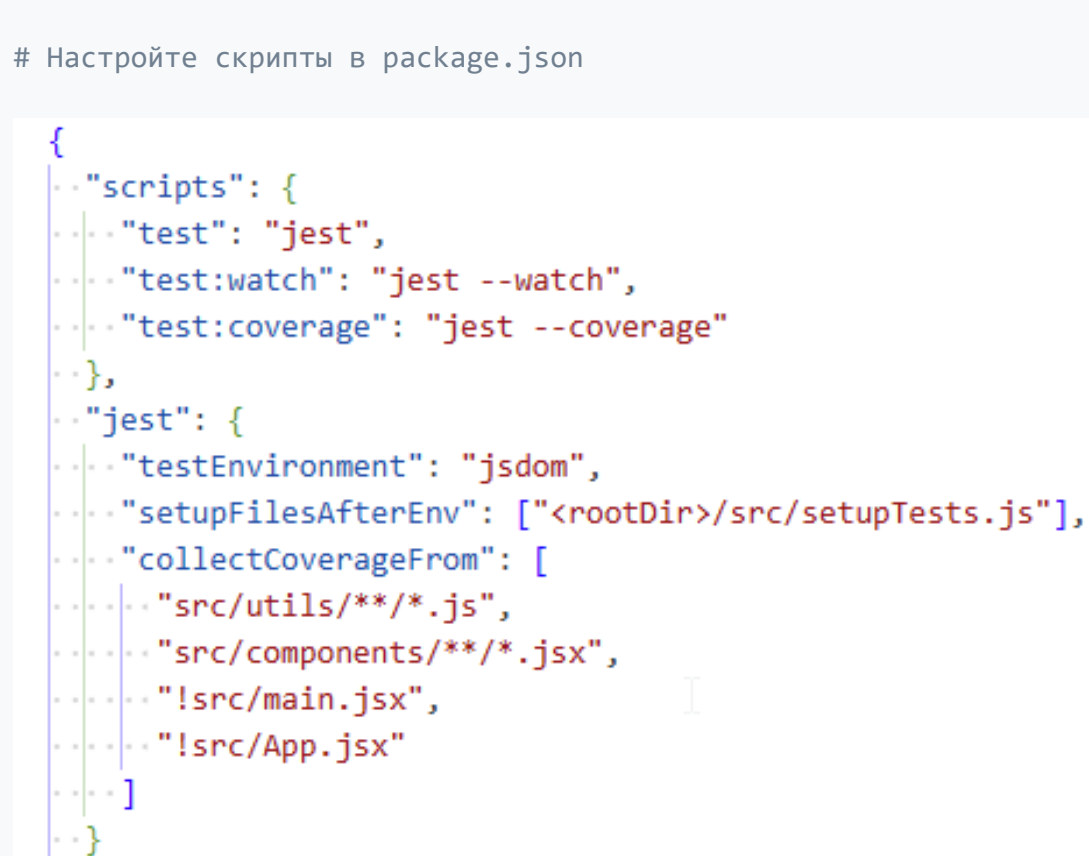
- Настроить окружение для unit-тестирования с Jest
- Написать тесты для чистых JavaScript функций
- Освоить основы тестирования React компонентов
- Изучить инструменты разработчика для отладки
- Научиться анализировать производительность с помощью Lighthouse

ПОРЯДОК ВЫПОЛНЕНИЯ РАБОТЫ

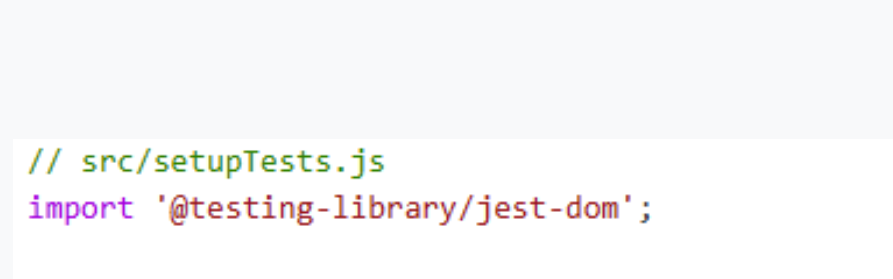
ЧАСТЬ 1: НАСТРОЙКА ОКРУЖЕНИЯ ДЛЯ ТЕСТИРОВАНИЯ

Объяснение: Jest - это фреймворк для тестирования JavaScript кода. Мы настроим его для тестирования утилит и простых компонентов.

```
# Перейдите в рабочую ветку
git checkout main
git pull origin main
git checkout -b feature/frontend-testing

# Установите зависимости для тестирования
cd my-react-app
npm install --save-dev jest @testing-library/react @testing-library/jest-dom jest-environment-jsdom

# Настройте скрипты в package.json
```



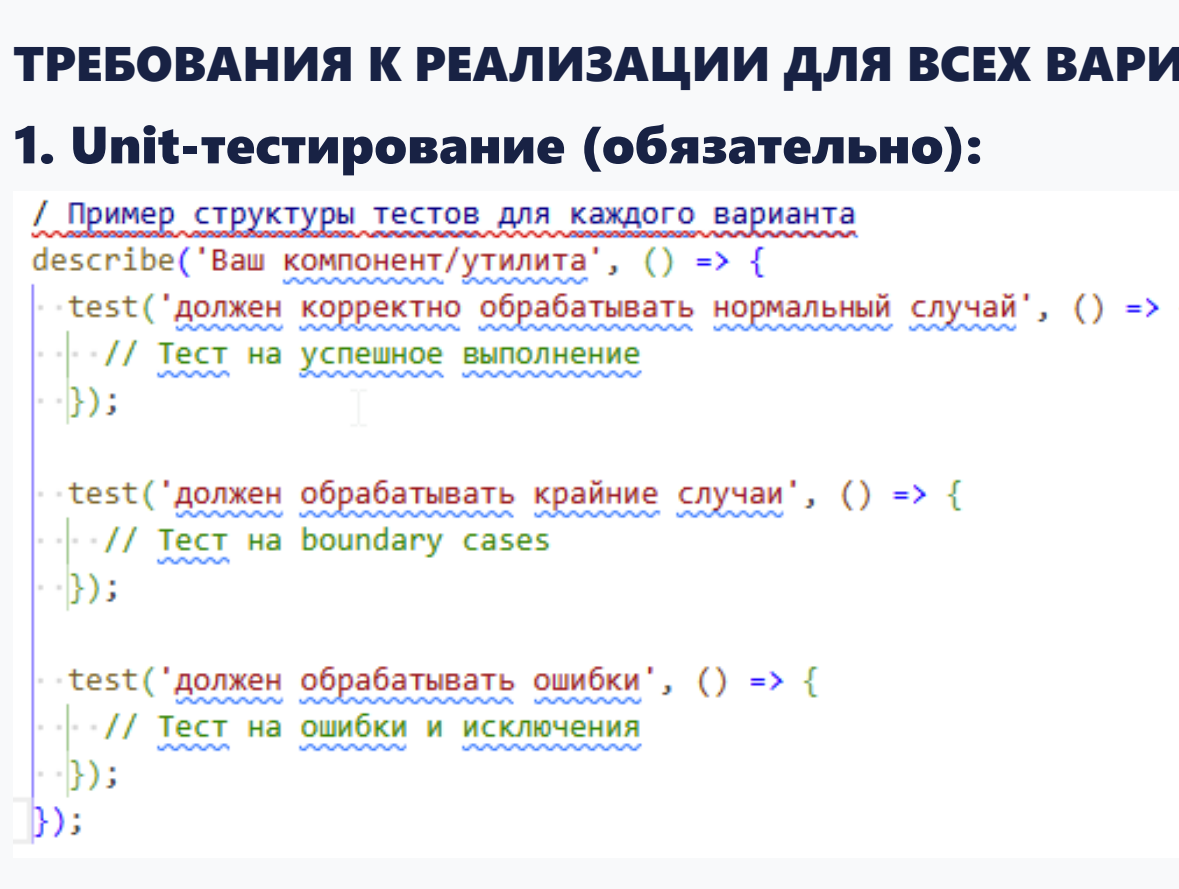
```
{
  "scripts": {
    "test": "jest",
    "test:watch": "jest --watch",
    "test:coverage": "jest --coverage"
  },
  "jest": {
    "testEnvironment": "jsdom",
    "setupFilesAfterEnv": ["<rootDir>/src/setupTests.js"],
    "collectCoverageFrom": [
      "src/utils/**/*.js",
      "src/components/**/*.jsx",
      "!src/main.jsx",
      "!src/App.jsx"
    ]
  }
}
```



```
// src/setupTests.js
import '@testing-library/jest-dom';

// Моки для CSS файлов
jest.mock('./App.css', () => ({}));
jest.mock('./components/Header.css', () => ({}));
```

ТРЕБОВАНИЯ К РЕАЛИЗАЦИИ ДЛЯ ВСЕХ ВАРИАНТОВ

1. Unit-тестирование (обязательно):



```
/ Пример структуры тестов для каждого варианта
describe('Ваш компонент/утилита', () => {
  test('должен корректно обрабатывать нормальный случай', () => {
    // Тест на успешное выполнение
  });

  test('должен обрабатывать крайние случаи', () => {
    // Тест на boundary cases
  });

  test('должен обрабатывать ошибки', () => {
    // Тест на ошибки и исключения
  });
});
```

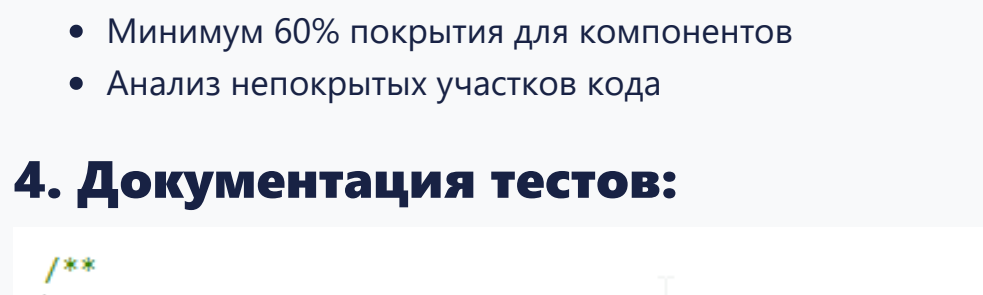
2. Инструменты разработчика (обязательно):

- **Console:** Использование `console.table()`, `console.group()`, логирование
- **Sources:** Установка breakpoints, пошаговая отладка
- **Network:** Анализ запросов, оптимизация загрузки
- **Performance:** Запись и анализ производительности
- **Lighthouse:** Полный аудит с рекомендациями

3. Анализ покрытия (обязательно):

- Минимум 70% покрытия для утилит
- Минимум 60% покрытия для компонентов
- Анализ непокрытых участков кода

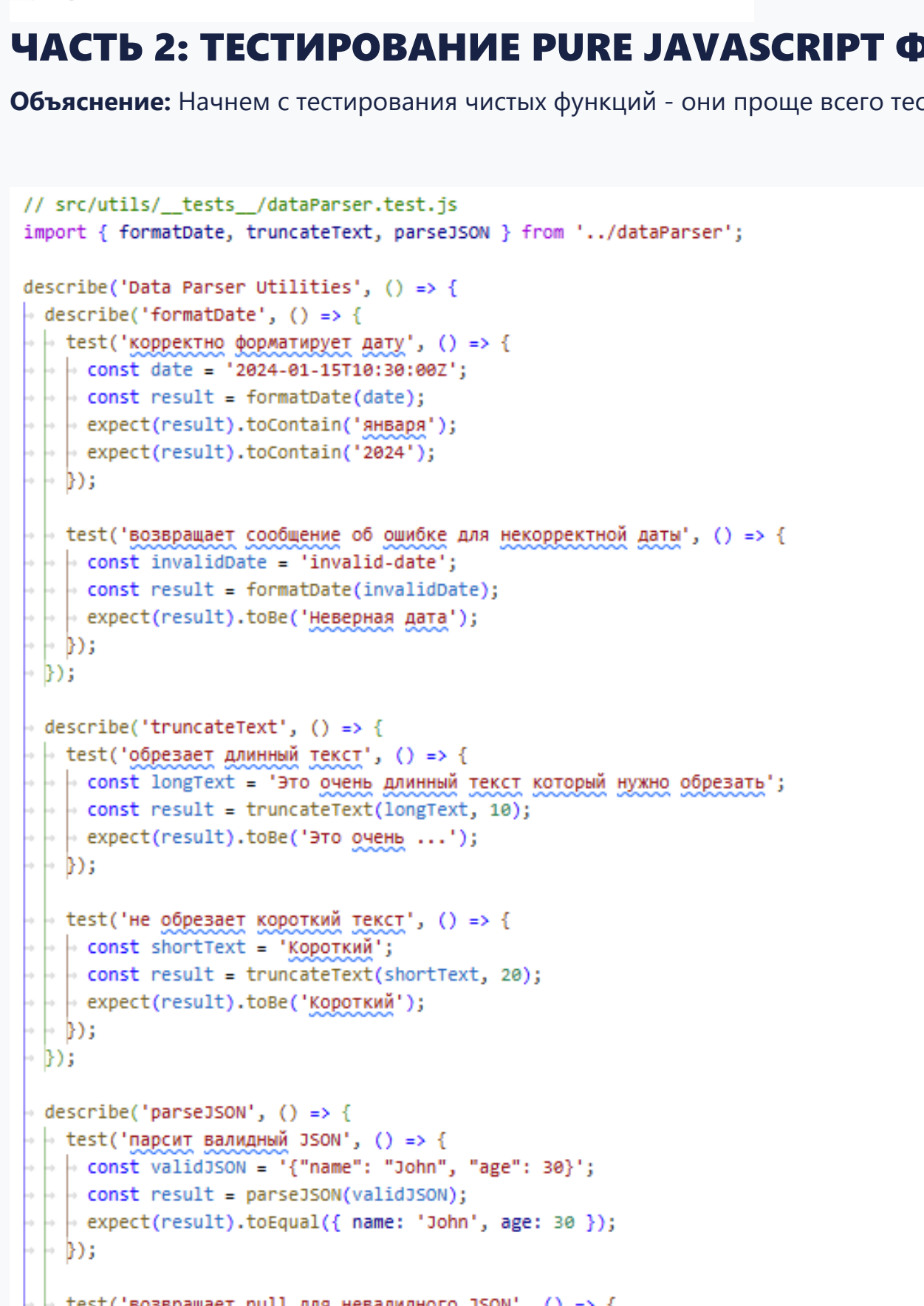
4. Документация тестов:



```
/**
 * @description Тестирует функцию форматирования даты
 * @case Нормальная дата
 * @case Некорректная дата
 * @case Пустая строка
 */
```

ЧАСТЬ 2: ТЕСТИРОВАНИЕ PURE JAVASCRIPT ФУНКЦИЙ

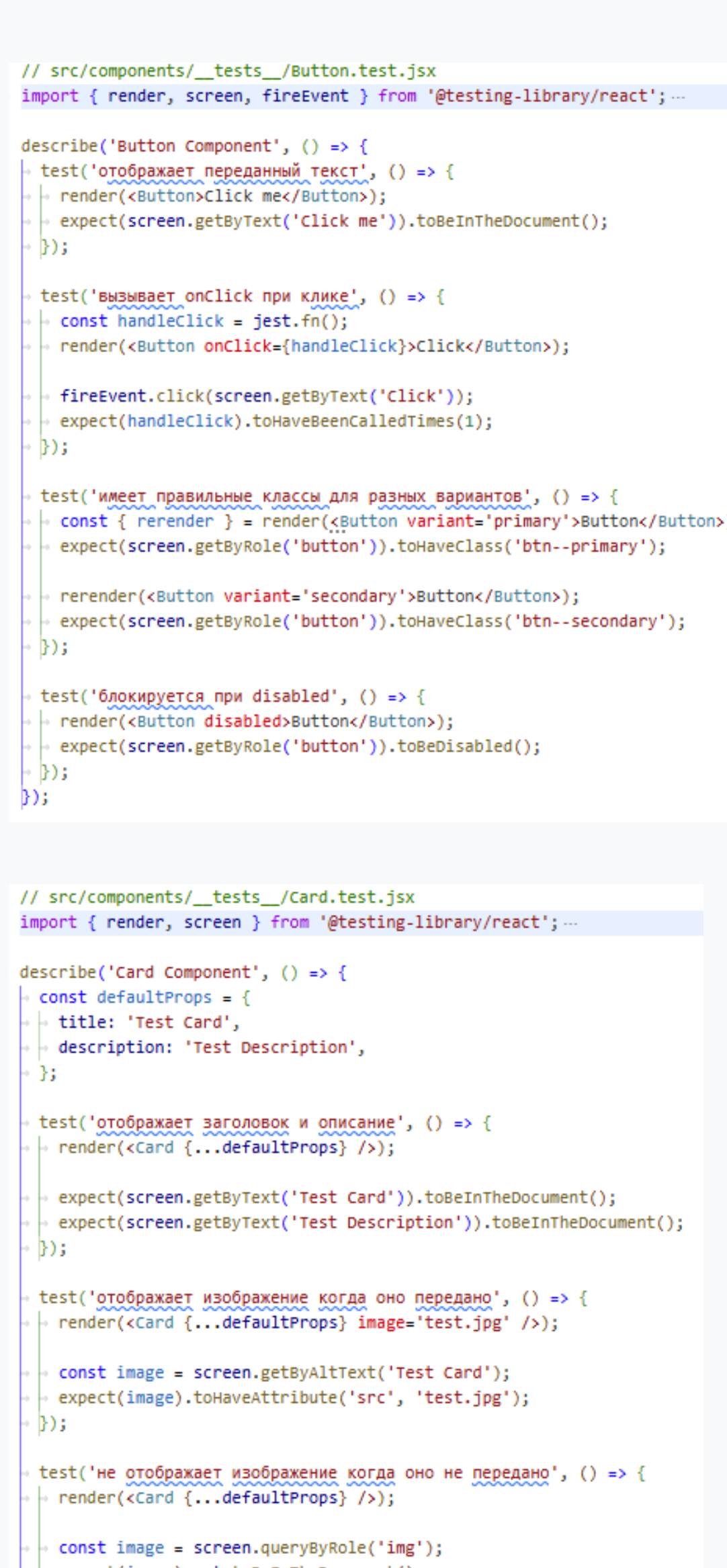
Объяснение: Начнем с тестирования чистых функций - они проще всего тестируются и не зависят от React.



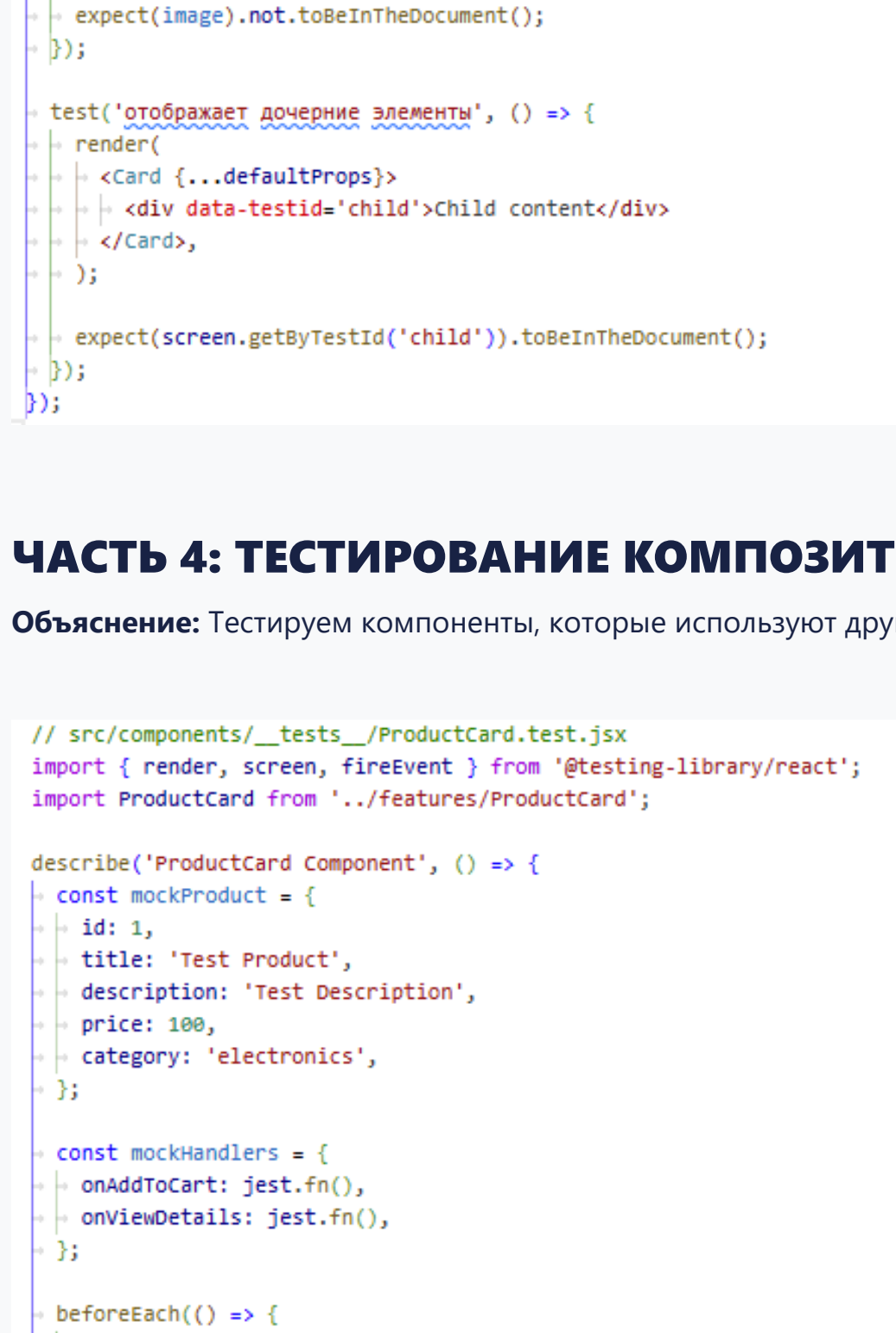
```
// src/utils/__tests__/dataParser.test.js
import { formatDate, truncateText, parseJSON } from '../dataParser';

describe('Data Parser Utilities', () => {
  describe('formatDate', () => {
    test('корректно форматирует дату', () => {
      const date = '2024-01-15T10:30:00Z';
      const result = formatDate(date);
      expect(result).toContain('января');
      expect(result).toContain('2024');
    });

    test('возвращает сообщение об ошибке для некорректной даты', () => {
      const invalidDate = 'invalid-date';
      const result = formatDate(invalidDate);
      expect(result).toBe('Неверная дата');
    });
  });

  describe('truncateText', () => {
    test('обрезает длинный текст', () => {
      const longText = 'Это очень длинный текст который нужно обрезать';
      const result = truncateText(longText, 10);
      expect(result).toBe('Это очень ...');
    });

    test('не обрезает короткий текст', () => {
      const shortText = 'Короткий';
      const result = truncateText(shortText, 20);
      expect(result).toBe('Короткий');
    });
  });

  describe('parseJSON', () => {
    test('парсит валидный JSON', () => {
      const validJSON = '{"name": "John", "age": 30}';
      const result = parseJSON(validJSON);
      expect(result).toEqual({ name: 'John', age: 30 });
    });

    test('возвращает null для невалидного JSON', () => {
      const invalidJSON = '{invalid json}';
      const result = parseJSON(invalidJSON);
      expect(result).toBeNull();
    });
  });
});
```

ЧАСТЬ 3: ТЕСТИРОВАНИЕ ПРОСТЫХ REACT КОМПОНЕНТОВ

Объяснение: Тестируем презентационные компоненты из ЛР7, которые не имеют состояния.



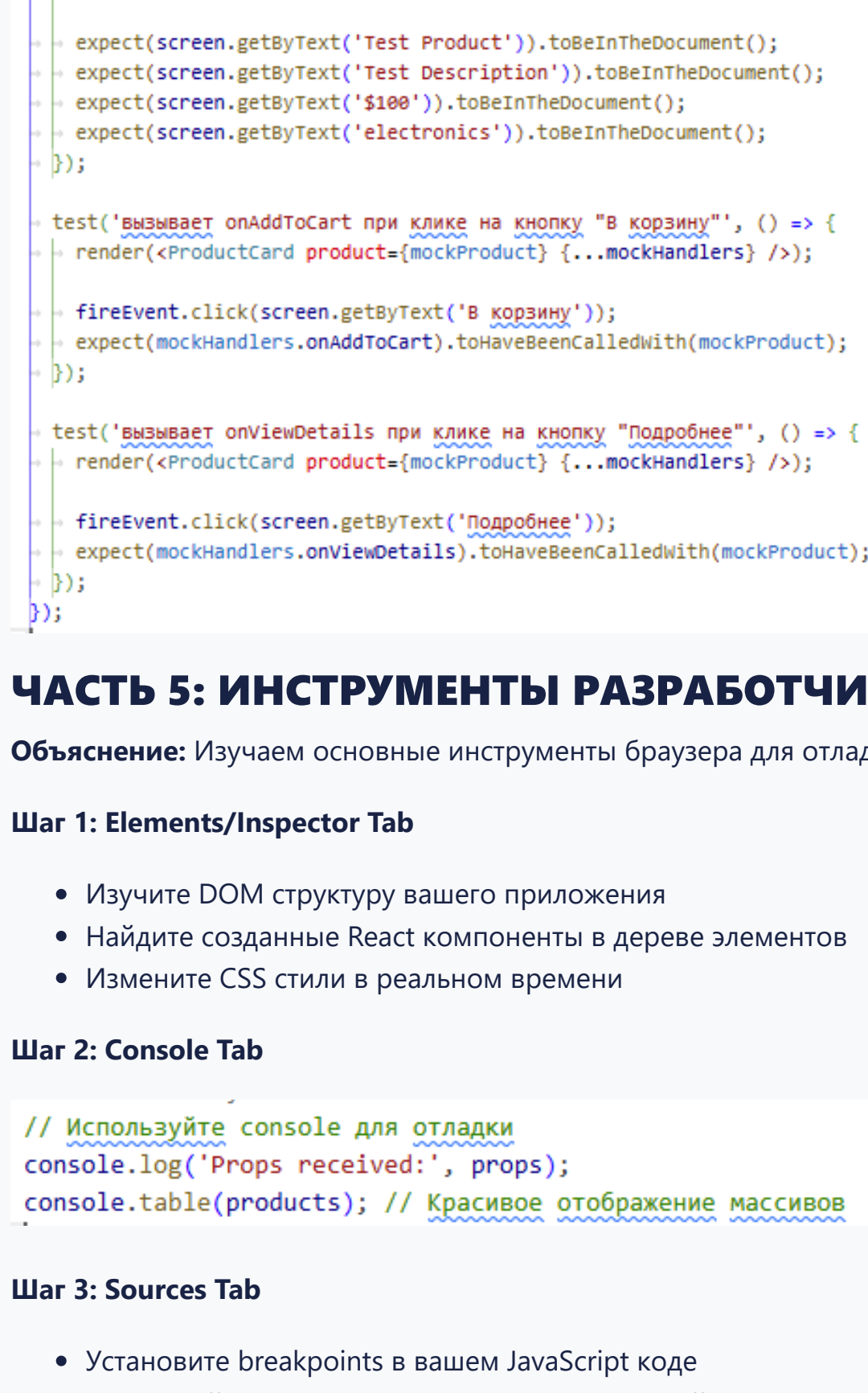
```
// src/components/__tests__/Button.test.jsx
import { render, screen, fireEvent } from '@testing-library/react';...

describe('Button Component', () => {
  test('отображает переданный текст', () => {
    render(<Button>Click me</Button>);
    expect(screen.getByText('Click me')).toBeInTheDocument();
  });

  test('вызывает onClick при клике', () => {
    const handleClick = jest.fn();
    render(<Button onClick={handleClick}>Click</Button>);

    fireEvent.click(screen.getByText('Click'));
    expect(handleClick).toHaveBeenCalledTimes(1);
  });

  test('имеет правильные классы для разных вариантов', () => {
    const { rerender } = render(<Button variant='primary'>Button</Button>);
    expect(screen.getByRole('button')).toHaveClass('btn--primary');

    rerender(<Button variant='secondary'>Button</Button>);
    expect(screen.getByRole('button')).toHaveClass('btn--secondary');
  });

  test('блокируется при disabled', () => {
    render(<Button disabled>Button</Button>);
    expect(screen.getByRole('button')).toBeDisabled();
  });
});
```



```
// src/components/__tests__/Card.test.jsx
import { render, screen } from '@testing-library/react';...

describe('Card Component', () => {
  const defaultProps = {
    title: 'Test Card',
    description: 'Test Description',
  };

  test('отображает заголовок и описание', () => {
    render(<Card {...defaultProps} />);

    expect(screen.getByText('Test Card')).toBeInTheDocument();
    expect(screen.getByText('Test Description')).toBeInTheDocument();
  });

  test('отображает изображение когда оно передано', () => {
    render(<Card {...defaultProps} image='test.jpg' />);

    const image = screen.getByAltText('Test Card');
    expect(image).toHaveAttribute('src', 'test.jpg');
  });

  test('не отображает изображение когда оно не передано', () => {
    render(<Card {...defaultProps} />);

    const image = screen.queryByRole('img');
    expect(image).not.toBeInTheDocument();
  });

  test('отображает дочерние элементы', () => {
    render(
      <Card {...defaultProps}>
        <div data-testid='child'>Child content</div>
      </Card>,
    );

    expect(screen.getByTestId('child')).toBeInTheDocument();
  });
});
```

ЧАСТЬ 4: ТЕСТИРОВАНИЕ КОМПОЗИТНЫХ КОМПОНЕНТОВ

Объяснение: Тестируем компоненты, которые используют другие компоненты.



```
// src/components/__tests__/ProductCard.test.jsx
import { render, screen, fireEvent } from '@testing-library/react';
import ProductCard from '../features/ProductCard';

describe('ProductCard Component', () => {
  const mockProduct = {
    id: 1,
    title: 'Test Product',
    description: 'Test Description',
    price: 100,
    category: 'electronics',
  };

  const mockHandlers = {
    onAddToCart: jest.fn(),
    onViewDetails: jest.fn(),
  };

  beforeEach(() => {
    mockHandlers.onAddToCart.mockClear();
    mockHandlers.onViewDetails.mockClear();
  });

  test('отображает информацию о продукте', () => {
    render(<ProductCard product={mockProduct} {...mockHandlers} />);

    expect(screen.getByText('Test Product')).toBeInTheDocument();
    expect(screen.getByText('Test Description')).toBeInTheDocument();
    expect(screen.getByText('$100')).toBeInTheDocument();
    expect(screen.getByText('electronics')).toBeInTheDocument();
  });

  test('вызывает onAddToCart при клике на кнопку "В корзину"', () => {
    render(<ProductCard product={mockProduct} {...mockHandlers} />);

    fireEvent.click(screen.getByText('В корзину'));
    expect(mockHandlers.onAddToCart).toHaveBeenCalledWith(mockProduct);
  });

  test('вызывает onViewDetails при клике на кнопку "Подробнее"', () => {
    render(<ProductCard product={mockProduct} {...mockHandlers} />);

    fireEvent.click(screen.getByText('Подробнее'));
    expect(mockHandlers.onViewDetails).toHaveBeenCalledWith(mockProduct);
  });
});
```

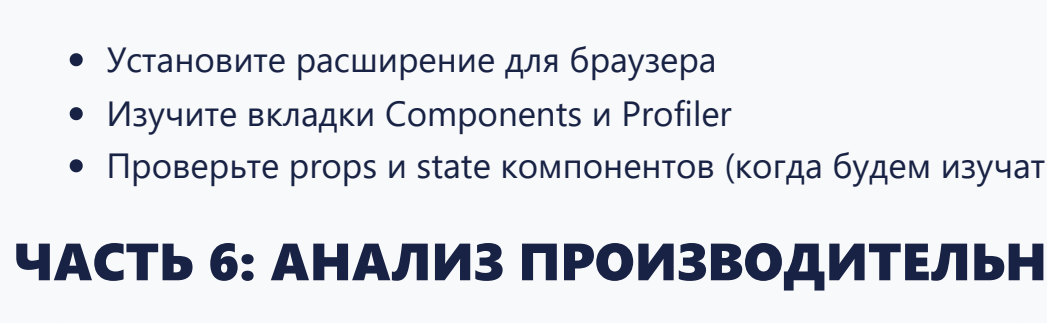
ЧАСТЬ 5: ИНСТРУМЕНТЫ РАЗРАБОТЧИКА ДЛЯ ОТЛАДКИ

Объяснение: Изучаем основные инструменты браузера для отладки приложений.

Шаг 1: Elements/Inspector Tab

- Изучите DOM структуру вашего приложения
- Найдите созданные React компоненты в дереве элементов
- Измените CSS стили в реальном времени

Шаг 2: Console Tab



```
// Используйте console для отладки
console.log('Props received:', props);
console.table(products); // Красивое отображение массивов
```

Шаг 3: Sources Tab

- Установите breakpoints в вашем JavaScript коде
- Используйте Step Over, Step Into для пошаговой отладки
- Наблюдайте за значениями переменных в Scope

Шаг 4: Network Tab

- Проанализируйте загрузку ресурсов
- Найдите медленно загружающиеся файлы
- Проверьте статусы HTTP запросов

Шаг 5: React Developer Tools

- Установите расширение для браузера
- Изучите вкладки Components и Profiler
- Проверьте props и state компонентов (когда будем изучать состояние)

ЧАСТЬ 6: АНАЛИЗ ПРОИЗВОДИТЕЛЬНОСТИ С LIGHTHOUSE

Объяснение: Lighthouse - инструмент для аудита качества веб-приложений.

Шаг 1: Запуск аудита

- Откройте вкладку Lighthouse в DevTools
- Выберите категории: Performance, Accessibility, Best Practices, SEO
- Запустите генерацию отчета

Шаг 2: Анализ метрик производительности

- **First Contentful Paint (FCP)** - когда появляется первый контент
- **Largest Contentful Paint (LCP)** - когда загружается самый большой элемент
- **Cumulative Layout Shift (CLS)** - стабильность layout

Шаг 3: Рекомендации по улучшению

- Оптимизация изображений
- Удаление неиспользуемого CSS/JS
- Включение кэширования

ЧАСТЬ 7: ЗАПУСК ТЕСТОВ И АНАЛИЗ РЕЗУЛЬТАТОВ

```
# Запуск всех тестов
npm test

# Запуск тестов в режиме наблюдения
npm run test:watch

# Запуск тестов с отчетом о покрытии
npm run test:coverage
```

Анализ отчета о покрытии:

- Откройте `coverage/lcov-report/index.html`
- Найдите непокрытые тестами участки кода
- Стремитесь к 70%+ покрытия для критической логики

ИНДИВИДУАЛЬНЫЕ ВАРИАНТЫ ЗАДАНИЙ - Для выбора номера варианта для бригады используется Генератор вариантов.

Общее задание для всех вариантов:
Создайте комплексную систему тестирования для вашего React-приложения. Реализуйте unit-тесты для утилит и ключевых компонентов, проанализируйте производительность с помощью инструментов разработчика и добейтесь минимум 70% покрытия кода тестами.

---

1 ПОДГРУППА (Варианты 1-8)

Вариант 1 - Новостной портал "NewsHub":

- **Unit-тесты для:** `formatDate`, `truncateText`, NewsCard, NewsList, NewsFilters
- **Проверка:** форматирование дат, обрезка текста, фильтрация новостей
- **Инструменты:** Анализ загрузки изображений в Network tab, аудит доступности контента

Вариант 2 - Интернет-магазин "TechStore":

- **Unit-тесты для:** `formatCurrency`, `calculateTotal`, ProductCard, ShoppingCart, ProductFilters
- **Проверка:** форматирование цен, расчет итогов, фильтрация товаров
- **Инструменты:** Профилирование производительности корзины, анализ размеров bundle

Вариант 3 - Задачник "TaskManager":

- **Unit-тесты для:** `generateId`, `sortTasks`, TaskItem, TaskList, FilterButtons
- **Проверка:** генерация ID, сортировка задач, фильтрация по статусу
- **Инструменты:** Отладка событий drag-and-drop, анализ перерисовок компонентов

Вариант 4 - Блог-платформа "BlogSpace":

- **Unit-тесты для:** `parseMarkdown`, `slugify`, PostCard, AuthorInfo, TagList
- **Проверка:** парсинг markdown, генерация slug, отображение тегов
- **Инструменты:** Анализ SEO с Lighthouse, проверка мета-тегов

Вариант 5 - Каталог фильмов "MovieCatalog":

- **Unit-тесты для:** `formatDuration`, `calculateRating`, MovieCard, GenreFilter, RatingStars
- **Проверка:** форматирование длительности, расчет рейтинга, фильтрация по жанрам
- **Инструменты:** Анализ загрузки трейлеров, производительность слайдеров

Вариант 6 - Погодное приложение "WeatherApp":

- **Unit-тесты для:** `convertTemperature`, `formatWindSpeed`, WeatherCard, ForecastList, CitySearch
- **Проверка:** конвертация температур, форматирование скорости ветра, поиск городов
- **Инструменты:** Анализ геолокации, производительность анимаций погоды

Вариант 7 - Социальная сеть "SocialNetwork":

- **Unit-тесты для:** `formatNumber`, `timeAgo`, PostCard, UserProfile, LikeButton
- **Проверка:** форматирование чисел (лайков), относительное время, взаимодействия
- **Инструменты:** Профилирование бесконечного скролла, анализ утечек памяти

Вариант 8 - Образовательная платформа "EduPlatform":

- **Unit-тесты для:** `calculateProgress`, `validateQuiz`, CourseCard, ProgressBar, QuizComponent
- **Проверка:** расчет прогресса, валидация тестов, отображение курсов
- **Инструменты:** Анализ доступности для скринридеров, производительность видеоплеера

---

2 ПОДГРУППА (Варианты 9-16)

Вариант 9 - Ресторанное приложение "FoodDelivery":

- **Unit-тесты для:** `formatTime`, `calculateOrderTotal`, DishCard, MenuSection, CartItem
- **Проверка:** форматирование времени приготовления, расчет суммы заказа, управление корзиной
- **Инструменты:** Анализ производительности модальных окон, отладка форм заказа

Вариант 10 - Библиотека "DigitalLibrary":

- **Unit-тесты для:** `searchBooks`, `filterByGenre`, BookCard, SearchResults, Bookshelf
- **Проверка:** алгоритмы поиска, фильтрация по жанрам, управление коллекцией
- **Инструменты:** Профилирование поиска по большому каталогу, анализ индексации

Вариант 11 - Фитнес-трекер "FitTrack":

- **Unit-тесты для:** `calculateCalories`, `formatWorkoutTime`, ExerciseCard, WorkoutTimer, ProgressChart
- **Проверка:** расчет калорий, форматирование времени тренировок, отображение прогресса
- **Инструменты:** Анализ производительности таймеров, отладка Web Workers

Вариант 12 - Портфолио фотографа "PhotoGallery":

- **Unit-тесты для:** `optimizeImageUrl`, `sortPhotos`, PhotoCard, GalleryGrid, Lightbox
- **Проверка:** оптимизация URL изображений, сортировка фото, работа галереи
- **Инструменты:** Анализ загрузки изображений, производительность лайтбокса

Вариант 13 - Путешествия "TravelPlanner":

- **Unit-тесты для:** `calculateDistance`, `formatPrice`, DestinationCard, TripBuilder, MapComponent
- **Проверка:** расчет расстояний, форматирование цен, построение маршрутов
- **Инструменты:** Профилирование карт, анализ взаимодействия с геоданными

Вариант 14 - Музыкальный плеер "MusicStream":

- **Unit-тесты для:** `formatTime`, `shuffleArray`, TrackItem, Playlist, AudioPlayer
- **Проверка:** форматирование времени треков, алгоритм перемешивания, управление плейлистом
- **Инструменты:** Анализ Web Audio API, производительность визуализатора

Вариант 15 - Финтех "BudgetTracker":

- **Unit-тесты для:** `calculateBalance`, `categorizeExpense`, TransactionItem, BudgetChart, ReportGenerator
- **Проверка:** расчет баланса, категоризация расходов, генерация отчетов
- **Инструменты:** Профилирование графиков, анализ точности расчетов

Вариант 16 - Ивенты "EventManager":

- **Unit-тесты для:** `formatEventDate`, `checkAvailability`, EventCard, Calendar, RegistrationForm
- **Проверка:** форматирование дат событий, проверка доступности, работа календаря
- **Инструменты:** Анализ производительности календаря, отладка валидации форм

КОНТРОЛЬНЫЕ ВОПРОСЫ

1. **Какие виды тестирования вы применили и в чем их различия?** Приведите примеры unit-тестов из вашего проекта.
2. **Как Jest помогает в организации тестов?** Какие функции Jest вы использовали (describe, test, expect и др.)?
3. **Какие инструменты разработчика оказались наиболее полезными?** Приведите пример решения конкретной проблемы с помощью DevTools.
4. **Какие метрики Lighthouse важны для пользовательского опыта?** Как вы можете улучшить эти метрики в своем приложении?
5. **Что такое покрытие кода и почему оно важно?** Какой процент покрытия вы достигли и какие участки кода остались непокрытыми?

Добавить ответ на задание

Состояние ответа

Номер попытки	Попытка 1.
Состояние ответа на задание	Ответы на задание еще не представлены
Состояние оценивания	Не оценено
Оставшееся время	19 дн. 14 час. осталось
Последнее изменение	-
Комментарии к ответу	› Комментарии (0)